

任务目标

完成 NexusCRM 的多角色登录基础能力。登录页需要通过 Axios 请求 MSW 接口，登录成功后由 Pinia 保存 Token、用户资料、动态菜单和权限编码。

完成后的效果

输入正确账号和密码后可以进入首页工作台。输入错误密码会显示错误提示。刷新页面后，登录状态可以通过本地 Token 和 MSW 用户资料接口恢复。

你需要完成什么

1. 扩展模拟用户、角色、菜单和权限数据。
2. 实现登录、退出、用户资料、菜单、权限接口。
3. 封装认证 API。
4. 创建用户 Pinia Store。
5. 改造登录页面，让登录动作走接口请求。

请求链路

本任务的核心链路是：

```
LoginView 登录表单
-> src/api/auth.js
-> src/api/request.js
-> MSW /api/auth/login
-> userStore 保存 Token
-> /api/auth/profile、/api/auth/routes、/api/auth/permissions
```

页面不要直接导入模拟账号数组，也不要点击按钮后直接 `router.push('/dashboard')`。

目录结构要求

任务完成后至少应有以下结构：

```
src/
├── api/
│   ├── auth.js
│   └── request.js
├── mock/
│   ├── database/
│   │   ├── seed.js
│   │   └── store.js
│   └── handlers/
│       └── index.js
```

```
├── stores/
│   ├── index.js
│   └── user.js
└── views/
    ├── login/
    │   └── LoginView.vue
```

操作步骤

步骤 1：准备固定演示账号

在 `src/mock/database/seed.js` 中准备 5 个固定账号：

账号	密码	角色
admin	Admin@2026	admin
manager	Manager@2026	manager
sales	Sales@2026	sales
support	Support@2026	support
viewer	Viewer@2026	viewer

每个用户至少包含：

- `id`
- `username`
- `name`
- `email`
- `phone`
- `avatar`
- `role`
- `roleName`
- `status`
- `lastLoginAt`

可以额外生成 10 个普通用户，供后续系统管理页面使用。

步骤 2：创建角色、菜单和权限数据

建议创建：

- `roles`：角色列表和权限编码。
- `menus`：菜单列表，包含 `path`、`title`、`shortLabel`、`roles`。
- `sessions`：模拟登录会话列表。

至少准备以下权限编码：

```
dashboard:view
customer:view
customer:create
opportunity:create
contract:approve
ticket:handle
```

步骤 3：实现认证相关 MSW 接口

在 `src/mock/handlers/index.js` 中新增：

方法	地址	说明
POST	<code>/api/auth/login</code>	校验账号密码，返回 Token
POST	<code>/api/auth/logout</code>	注销当前 Token
GET	<code>/api/auth/profile</code>	返回当前用户资料
GET	<code>/api/auth/routes</code>	返回当前角色菜单
GET	<code>/api/auth/permissions</code>	返回当前角色按钮权限

统一响应格式仍然使用：

```
{
  code: 0,
  message: '操作成功',
  data: {}
}
```

步骤 4：封装认证 API

在 `src/api/auth.js` 中封装：

- `login(credentials)`
- `logout()`
- `getProfile()`
- `getAuthRoutes()`
- `getPermissions()`

页面和 Store 不直接写 Axios 路径，统一调用 API 方法。

步骤 5：创建用户 Store

在 `src/stores/user.js` 中保存：

- `token`

- user
- menus
- permissions

并实现：

- login(credentials)
- restoreSession()
- logout()
- clearSession()
- fetchProfile()
- fetchMenus()
- fetchPermissions()

登录成功后，先保存 Token，再请求用户资料、菜单和权限。

步骤 6：改造登录页面

登录页需要完成：

1. 账号和密码表单绑定。
2. Element Plus 表单校验。
3. 登录按钮 loading 状态。
4. 登录失败显示错误提示。
5. 登录成功跳转到 redirect 或 /dashboard。
6. 登录失败后清空密码输入框。

自测步骤

1. 启动项目。
2. 访问 /login。
3. 输入 admin / Admin@2026，确认登录成功。
4. 退出或清理状态后，再输入错误密码，确认显示“账号或密码错误”。
5. 登录成功后刷新页面，确认仍能恢复用户资料。
6. 打开浏览器 Network 或接口文档，确认登录页请求了 MSW 接口。

验收清单

- ☐ 登录页不是直接跳转，而是调用登录接口。
- ☐ 正确账号可以登录。
- ☐ 错误密码会显示提示。
- ☐ Token 保存到了用户 Store 和本地存储。
- ☐ 登录成功后能请求用户资料、菜单和权限。
- ☐ 登录失败不会进入后台。

- ☐ 刷新页面后用户状态可以恢复。
- ☐ 用户响应中不应暴露明文密码。

易错提醒

常见错误	现象	解决方式
登录页直接 <code>router.push</code>	错误密码也能进后台	改为调用 <code>userStore.login()</code>
用户 Store 只保存 Token	页面无法显示用户信息	登录后继续请求 <code>profile</code>
忘记持久化 Token	刷新后掉登录	使用 <code>localStorage</code> 保存 Token
页面直接导入模拟账号	不像真实前后端交互	统一走 <code>Axios + MSW</code>

任务内选做

1. 使用 PBKDF2 或其他摘要方式模拟密码校验，避免模拟数据库保存明文密码。
2. 登录成功后不要弹出敏感信息，只做简单成功提示或直接跳转。
3. 增加禁用账号测试，禁用账号登录返回 403。